SNOWPRO ADVANCED ADMINISTRATOR (ADA-C02)

MODULE 3: DATA AND OBJECT MANAGEMENT (15% EXAM WEIGHT)

EXECUTIVE STUDY ROADMAP

Module 3 accounts for **15% of the SnowPro Advanced Administrator (ADA-C02) exam**. This domain evaluates an administrator's capacity to optimize compute resource allocation, configure multi-cluster warehouses, manage diverse table types, orchestrate serverless ingestion pipelines, and enforce storage cost optimization. Mastery of virtual warehouse auto-suspend/scaling policies, table Time Travel vs. Fail-safe mechanics, Snowpipe auto-ingestion, and Streams/Tasks DAG workflows is mandatory to pass this domain.

1. VIRTUAL WAREHOUSE MANAGEMENT & COMPUTE GOVERNANCE

Warehouse Sizing & Scaling Architecture

Snowflake virtual warehouses provide isolated compute resources. Warehouses scale *vertically* (sizing from XS to 6X-Large) to handle query complexity, and *horizontally* (multi-cluster scaling) to manage concurrent query throughput. Each sizing tier doubles the compute capacity (number of servers per cluster) and doubles credit consumption per hour.

Warehouse Size	Servers per Cluster	Credits / Hour	Primary Use Case
X-Small (XS)	1	1	Development, light ELT, low-concurrency tasks
Small (S)	2	2	Small team ad-hoc queries & BI reporting
Medium (M)	4	4	Standard production workloads, automated ETL
Large (L)	8	8	Complex joins, large aggregations, high memory
X-Large (XL)	16	16	Enterprise data warehouse loads & data science
2X-Large to 6X-Large	32 to 512	32 to 512	Massive parallel batch processing & heavy transformations

Multi-Cluster Warehouses & Scaling Policies

Multi-cluster warehouses automatically spin up additional clusters (up to `MAX_CLUSTERS = 10`) to handle concurrency spikes. Administrators configure two primary scaling policies:

Scaling Policy	Cluster Startup Trigger	Cluster Shutdown Behavior	Best For
Standard (Default)	Spins up an additional cluster immediately when queries queue up.	Shuts down cluster after 2 consecutive checks show no queued queries.	Strict SLA environments prioritizing execution speed over credit minimization.
Economy	Waits until estimated queue processing time exceeds 6 minutes of compute workload.	Conserves credits by delaying new cluster provisioning until queue is significant.	Cost-sensitive batch workloads where query latency is tolerable.

> **ADA-C02 EXAM KEY RULE:** Credit billing for virtual warehouses accrues on a per-second basis with a **60-second minimum charge** each time a warehouse starts or resumes. Setting `AUTO_SUSPEND = 60` for ad-hoc warehouses prevents unnecessary credit drain, while `AUTO_RESUME = TRUE` ensures zero downtime for users.

Resource Monitors & Credit Controls

Resource Monitors track credit consumption at either the **Account level** or **Warehouse level**. When specified credit thresholds (percentages of quota) are met, Snowflake executes defined actions:

- **NOTIFY:** Sends email alerts to account administrators without interrupting active workloads.
- **SUSPEND:** Prevents new queries from starting, but allows currently running queries to finish executing.
- **SUSPEND_IMMEDIATE:** Immediately cancels all currently running queries and halts the virtual warehouse instantly.

2. TABLE TYPES, STORAGE & LIFECYCLE MANAGEMENT

Snowflake Table Types Matrix

Selecting the correct table type is crucial for balancing query performance, Time Travel recovery, and storage costs. The ADA-C02 exam heavily tests the operational boundaries of Permanent, Transient, Temporary, External, and Iceberg tables.

Table Type	Scope / Lifetime	Time Travel Range	Fail-safe Period	Storage Cost Impact
Permanent	Persistent database object	0 to 90 Days (Default: 1)	7 Days (Mandatory)	Highest (Base + Time Travel + Fail-safe)

Table Type	Scope / Lifetime	Time Travel Range	Fail-safe Period	Storage Cost Impact
Transient	Persistent database object	0 to 1 Day (Default: 1)	0 Days (No Fail-safe)	Moderate (Base + 1-Day Time Travel max)
Temporary	Session-scoped (auto-dropped)	0 to 1 Day (Default: 1)	0 Days (No Fail-safe)	Lowest (Dropped when session ends)
External	Read-only pointer to cloud stage	N/A (No Time Travel)	N/A (No Fail-safe)	Zero Snowflake storage (Billed by cloud provider)
Iceberg	Apache Iceberg open table spec	0 to 90 Days (Snowflake catalog)	7 Days (External/Snowflake)	Custom catalog or Snowflake storage

CRITICAL FAIL-SAFE RULE: Transient and Temporary tables DO NOT have a Fail-safe period. If a transient table is dropped or modified after its Time Travel window expires, the data is permanently unrecoverable. Snowflake Support cannot assist in recovering transient or temporary tables from Fail-safe.

3. DATA STAGING, LOADING & UNLOADING ARCHITECTURE

Internal vs. External Stages

Data staging stores raw data files prior to loading into Snowflake tables, or receives exported files during data unloading. Snowflake supports three internal stage variants and external stages pointing to cloud storage (AWS S3, Azure Blob, GCP GCS).

Stage Type	SQL Syntax	Scope & Accessibility	Best Practice Use Case
User Stage	@~	Scoped strictly to a single user. Cannot be shared with other users.	Personal testing and temporary script loading
Table Stage	@%table_name	Scoped to a specific table. Accessible by anyone with table privileges.	Direct single-table loading operations
Named Internal Stage	@named_stage	Database object with explicit RBAC privileges.	Multi-table pipelines & central staging repositories
Named External Stage	@ext_stage	Points to external S3/GCS/Azure URI using Storage Integration.	Production enterprise cloud data pipelines

Snowpipe Automated Continuous Ingestion

Snowpipe provides automated micro-batch data loading using serverless compute (managed by Snowflake). Snowpipe detects newly staged files via cloud messaging services (AWS SQS, Azure Event Grid, GCP

Pub/Sub) or REST API calls.

- **Serverless Compute:** Snowpipe compute is independent of virtual warehouses. Billed strictly per file loaded + serverless execution time.
- **`AUTO_INGEST = TRUE`:** Enables automatic event notification handling via cloud storage event queues.
- **`PIPE_EXECUTION_PAUSED = TRUE`:** Temporarily halts Snowpipe execution during maintenance without losing queued file messages.
- **Load History Retention:** Snowpipe tracks loaded files in metadata for 14 days (viewable via `COPY_HISTORY` function).

4. CONTINUOUS PIPELINES: STREAMS AND TASKS

Table Streams (Change Data Capture)

A Stream captures Change Data Capture (CDC) delta records (inserts, updates, deletes) committed to a source table. Streams create metadata columns: `METADATA$ACTION` (INSERT/DELETE), `METADATA$ISUPDATE` (TRUE/FALSE), and `METADATA$ROW_ID`.

Task Scheduling & Task Graphs (DAGs)

Tasks execute single SQL statements or stored procedures on a defined schedule or event trigger. Tasks can be executed using a user-managed virtual warehouse or **Serverless compute** managed by Snowflake. A root task initiates a Directed Acyclic Graph (DAG) using the `AFTER` clause for child tasks.

> **PIPELINE COST OPTIMIZATION TRICK:** To prevent empty task runs when no new data has arrived in a stream, add a condition check to the Task definition:
> WHEN SYSTEM$STREAM_HAS_DATA('my_stream')
> If the stream contains no CDC deltas, the task skips execution completely, incurring zero compute cost.

5. HANDS-ON PRACTICE LAB WORKSHEETS

Lab 1: Multi-Cluster Warehouse & Resource Monitor Setup

```
-- Create a Resource Monitor with alerts and hard suspension CREATE OR REPLACE RESOURCE MONITOR
rm_prod_governance WITH CREDIT_QUOTA = 500 FREQUENCY = MONTHLY START_TIMESTAMP = IMMEDIATELY TRIGGERS
ON 80% DO NOTIFY ON 100% DO SUSPEND ON 110% DO SUSPEND_IMMEDIATE; -- Create an Auto-scaling
Multi-Cluster Warehouse attached to Resource Monitor CREATE OR REPLACE WAREHOUSE wh_analytics_mc WITH
WAREHOUSE_SIZE = 'MEDIUM' MIN_CLUSTER_COUNT = 1 MAX_CLUSTER_COUNT = 5 SCALING_POLICY = 'STANDARD'
AUTO_SUSPEND = 120 AUTO_RESUME = TRUE RESOURCE_MONITOR = rm_prod_governance COMMENT = 'Multi-cluster
warehouse for analytics workloads';
```

Lab 2: Table Types & Time Travel Cost Management

```
-- Permanent Production Table with extended Time Travel (90 Days) CREATE OR REPLACE TABLE prod_db.sales.orders_perm ( order_id NUMBER, customer_id NUMBER, order_amount NUMBER(12,2), order_date DATE ) DATA_RETENTION_TIME_IN_DAYS = 90; -- Transient Staging Table (Max 1 day Time Travel, No Fail-safe) CREATE OR REPLACE TRANSIENT TABLE prod_db.sales.orders_stg ( order_id NUMBER, raw_payload VARIANT, ingested_at TIMESTAMP ) DATA_RETENTION_TIME_IN_DAYS = 1; -- Querying Historical Data via Time Travel SELECT * FROM prod_db.sales.orders_perm AT(OFFSET => -3600); -- 1 hour ago SELECT * FROM prod_db.sales.orders_perm BEFORE(TIMESTAMP => '2026-09-01 00:00:00'::timestamp);
```

Lab 3: Streams, Serverless Tasks, and CDC Ingestion DAG

```
-- 1. Create a Standard Table Stream on source staging table CREATE OR REPLACE STREAM sales_stg_stream ON TABLE prod_db.sales.orders_stg; -- 2. Create Root Task using Serverless Compute (Skips if Stream is Empty) CREATE OR REPLACE TASK task_process_orders USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'XSMALL' SCHEDULE = 'USING CRON */15 * * * * UTC' WHEN SYSTEM$STREAM_HAS_DATA('sales_stg_stream') AS INSERT INTO prod_db.sales.orders_perm (order_id, customer_id, order_amount, order_date) SELECT raw_payload:order_id::NUMBER, raw_payload:cust_id::NUMBER, raw_payload:amt::NUMBER(12,2), ingested_at::DATE FROM sales_stg_stream WHERE METADATA$ACTION = 'INSERT'; -- 3. Resume the Task Execution (Tasks are created in SUSPENDED state by default) ALTER TASK task_process_orders RESUME;
```